Information and Software Technology 179 (2025) 107644

Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof

# Locating requirements in backlog items: Content analysis and experiments with large language models

Ashley T. van Can *, Fabiano Dalpiaz

*Utrecht University, The Netherlands*

ARTICLE INFO

*Keywords:*
Agile requirements Engineering
User stories
Backlog items
Issue tracking systems
Content analysis
Large language models

ABSTRACT

**Context:** As agile development has become mainstream, requirements are increasingly managed via issue tracking systems (ITSs). These systems provide a single point of access to the product and sprint backlogs, bugs, ideas, and tasks for the development team. ITSs do not clearly separate requirements from work items.
**Objective:** We first tackle a *knowledge problem* concerning how requirements are formulated in ITSs, including their categorization and granularity, the presence of multiple requirements, and the existence of a motivation. Second, to assist practitioners in finding requirements in poorly organized ITSs without changing their way of working, we investigate the potential of automated techniques for identifying and classifying requirements in backlog items.
**Method:** Through quantitative content analysis, we analyze 1,636 product backlog items sampled from fourteen projects. To explore automated techniques for identifying requirements, we experiment with large language models (LLMs) due to their recent significance in NLP.
**Results:** The labeling of backlog items is largely inconsistent, and user-oriented functional requirements are the prevalent category. A backlog item often contains multiple requirements with different levels of granularity. The experiments with LLMs reveal that encoder-only models (BERT and RoBERTa) are most suitable for extracting and classifying requirements in backlog items compared to decoder-only models (Llama 3, Mistral 7B and ChatGPT with GPT 4).
**Conclusion:** We reveal knowledge and patterns about requirements documentation in ITSs, leading to a better empirical understanding of Agile RE. The experimental results with LLMs provide the foundation for developing automated, unobtrusive tools that identify and classify requirements in ITSs.

## 1. Introduction

Over the past few years, many software companies have transitioned to Agile development methods, often motivated by the promises of higher productivity, lower costs, heightened software quality and decreased time-to-market [1]. One core trait of the Agile manifesto, which we focus on in this paper, is the priority of building working software over high investments on comprehensive documentation [2].

In the context of requirements engineering (RE), thorough requirements specification requirements (often built in early stages and seldom updated) have been replaced by lightweight and incremental artifacts [3]. As indicated by the International Requirements Engineering Board (IREB) [4], agile projects tend to have only a few documents for representing requirements: product backlogs, sprint backlogs and task boards.

While authoring effort is reduced, the minimal documentation of requirements poses challenges related to traceability [5], possibly complicating tasks that rely on such information, such as tracing and software testing.

Previous research has explored and reviewed the benefits and challenges of agile RE [5] and the nature of the product backlog [6], but only limited work exists that studies how requirements are documented in real-world backlogs (generally stored in an issue tracking system, such as Jira[1]). To bridge this knowledge gap, we aim to analyze how requirements are documented within sprint and product backlog

* Corresponding author.
*E-mail addresses:* a.t.vancan@uu.nl (A.T. van Can), f.dalpiaz@uu.nl (F. Dalpiaz).

[1] https://www.atlassian.com/software/jira.

[2] We do not distinguish between whether items are in the product backlog or in the sprint backlog; we therefore use the term 'backlog items' to refer to the components of either backlog.

https://doi.org/10.1016/j.infsof.2024.107644
Received 4 July 2024; Received in revised form 29 September 2024; Accepted 5 December 2024
Available online 12 December 2024
0950-5849/© 2024 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

items,[2] for which we conduct a content analysis on 1,636 backlog items.

In our previous work [7], we conducted a preliminary analysis to examine how requirements are documented within backlog items. The results indicated an inconsistency in the backlog item labeling performed by practitioners and the difficulty of uniquely identifying requirements. In the first part of this paper, we report on a more accurate version of these results, which we obtained via an additional tagger and by the presentation of more nuanced statistics and answers to our research questions.

Given the informal nature of ITSs as documentation tools, we use the term *requirement* to denote a variety of textual fragments, without limiting ourselves to the use of specific templates such as the 'shall' format or user stories. This is in line with IREB's glossary [8], which defines a requirement as "A documented representation of a need, capability or property".

In the second part of the paper, motivated by the obtained evidence for the lack of an organized and systematic representation of requirements in backlog items, we investigate the potential of automated techniques for the identification of requirements in backlog items. We opt for an extraction mechanism, rather than designing a new method, due to (i) the goal of minimizing intervention on the team's practices, by proposing *unobtrusive* mechanisms; (ii) the existence of frameworks for organizing backlog items in the form of user stories [9,10].

Formerly, in the context of traditional specification documents, Abualhaija et al. [11] developed an automated approach for demarcating requirements from specifications. However, the informal nature of the product backlog [6] and the lack of standardized writing patterns call for the study of new techniques for extracting requirements from backlog items.

Within the broad landscape of technologies for developing automated tools–which include classical natural language processing, information retrieval, machine learning, deep learning–we focus on the most recent family of techniques: large language models (LLMs). LLMs are already the state-of-the-art for some automated tasks in RE [12]. We conduct experiments with both decoder-only models and with encoder-only architectures, such as BERT and RoBERTa, to compare the efficacy of these families of approaches.

This paper makes the following contributions:

- We provide a more accurate understanding (compared to [7]) of the challenges related to requirements documented in the backlog item, focusing on: (1) the correctness of backlog item labels chosen by practitioners, (2) the occurrence of multiple requirements in a single backlog item, and (3) the existence of justifications for requirements. The involvement of an additional tagger leads to a revised version of our dataset.
- We report on the performance of various LLMs for the task of automated identification of requirements in backlog items and for their classification into three classes: functional user-oriented, functional system-oriented, and non-functional.

The rest of the paper is structured as follows. Section 2 introduces our main research questions. Section 3 delves into our exploration, via content analysis, of how requirements are represented in backlog items. Section 4 reports on the second part of our research; the automated identification and classification of requirements via LLMs. Section 5 discusses the threats to validity, while Section 6 contrasts our research with related work. Finally, Section 7 draws conclusions and sketches future directions.

## 2. Research Questions

This study is concerned with understanding and addressing the challenges associated with the informal documentation of requirements in agile artifacts. Within this context, we define and refine two research questions.

### 2.1. RQ1: Documenting requirements in backlog items

RQ1 is a knowledge question on understanding the challenges associated with informal requirements documentation in agile development:

*RQ1 How are requirements documented in backlog items?*

To answer RQ1, we put forward three more specific research questions: RQ1.1 through RQ1.3.

*RQ1.1 To what extent do the backlog item labels chosen by practitioners reflect the requirements expressed in the items?*

RQ1.1 focuses on whether practitioners use suitable issue labels (e.g., 'Story' or 'Epic' in Jira) to distinguish requirements from backlog items that represent other aspects, such as bugs or tasks. In particular, we define one hypothesis that emerges when separating items with labels on requirements from items with labels that concern task execution: *H1: Backlog items with requirement-related labels contain requirements more often than backlog items with task-related labels.*

We then dig deeper into the understanding of the contents of the documented requirements by addressing RQ1.2:

*RQ1.2 What requirements documentation and refinement patterns can be found in backlog items?*

In the context of this research question, we study multiple facets of documenting and refining requirements. We first analyze the categories of requirements that are employed in the backlog items (see Table 2). Then, we investigate whether the requirements are uniquely identifiable [13], i.e., if a backlog item contains multiple requirements (note that, in ITSs, backlog items are the smallest identifiable piece of information). Finally, in case multiple requirements occur in the same backlog item, we analyze whether these pertain to multiple categories and if specific refinement patterns (e.g., user story to acceptance criteria) can be detected.

Given the importance of justifying the *why* in requirements engineering [14], we put forward RQ1.3 that explores whether the requirements in backlog items also include information concerning why they are needed.

*RQ1.3 To what extent are requirements complemented by a motivation for their existence?*

While addressing RQ1, we make use of collections of requirements from both open source (OSS) projects and proprietary ones, with the aim of *comparing* if OSS data, which are easier to retrieve and access, are aligned with how requirements are represented in proprietary projects, which are subject to confidentiality rules.

### 2.2. RQ2 : Locating and classifying requirements

The results from the first part of the research showed that the representation of requirements in backlog items, stored within ITSs, is often unsystematic (see Section 3.2 for all details). This led us to conducting research aiming to mitigate the identified challenges. We acknowledge that new methods and techniques for supporting the documentation and structuring the requirements could serve as a solution. In prior research, for example, we proposed the Quality User Story framework [9] that proposes criteria for writing high-quality user stories. Nevertheless, the empirical data from Part 1 of our research clearly shows that practitioners do not often adopt these methods. Consequently, we decided to opt for a different approach that works *unobtrusively*: the proposed technique should provide assistance without requiring the practitioner to change their way of working. In particular, we propose automated techniques for locating and classifying requirements, which have the potential of bringing clarity to poorly organized backlogs.

In the broad spectrum of the support technologies for automated support in software engineering, we decided to focus on the most recent

family of approaches: LLMs. They have emerged as a cornerstone in artificial intelligence, primarily due to their superior performance in interpreting and generating human language [15], and already constitute the state-of-the-art in several RE tasks [12]. We aim to investigate their performance in addressing the challenges associated with locating requirements information in product and sprint backlogs:

*RQ2. To what extent are language models capable of locating requirements in backlog items?*

Over the last decade, numerous (large) language models architectures have been introduced [15]. The architecture of these models can be distinguished into: encoder-only, decoder-only or hybrid [16]. In line with the guidelines by Vogelsang and Fischbach [17], we consider the first two categories here, as they are most appropriate for classification tasks. We leave experimenting with hybrid models to future work. We explore both the extraction and classification of requirements:

*RQ2.1 Do fine-tuned encoder-only outperform decoder-only models in terms of requirement extraction?*

*RQ2.2 Do fine-tuned encoder-only outperform decoder-only models in terms of requirements classification?*

## 3. Part 1: Content Analysis of Backlog Items

We address *RQ1: How are requirements documented in backlog items?* through the conduction of quantitative content analysis, a "research technique for making replicable and valid inferences from texts (or other meaningful matter) to the contexts of their use" [18].

### 3.1. Research Approach

In line with the distinction by White and Marsh [19], we use quantitative content analysis: after defining research questions and hypotheses, we collect data, determine the collection unit (issues organized into projects), define the coding scheme, tag the data, analyze, and write up the results.

One important deviation from the classic approach is that, while we started from literature knowledge to define basic codes (*deductive coding*), we have used a subset of the data to refine the coding scheme by introducing specific sub-categories (*inductive coding*). Our process is visualized in Fig. 1.

#### 3.1.1. Data collection and sampling

The topmost row of Fig. 1 illustrates how we collected data and we sampled the backlog items to analyze.

*Data collection.* We analyze data extracted from the ITSs of several projects, aiming to identify projects using the ITS as a backlog. We sampled OSS projects from two large public datasets [20,21]. Moreover, we collected seven internal development projects from a collaborating company: the low–code development platform provider Mendix.

From a quick scan, we could determine that many of the issue tracking systems for the open-source projects are employed mainly for reporting bugs rather than for documenting requirements. We automatically excluded projects with ≥80% issues labeled as 'bug' and <10% labeled as 'story'. This screening allowed us to filter out the projects whose dataset is likely not to serve as sprint or product backlog, retaining 88 projects.

In addition, since user stories (the main requirements artifact in Agile RE [22–24]) are user-oriented requirements, we decided to focus on projects with an evident user interaction component, eliminating projects without a clear user interface. The first author assessed the 88 retained projects based on their UI component and excluded irrelevant projects.

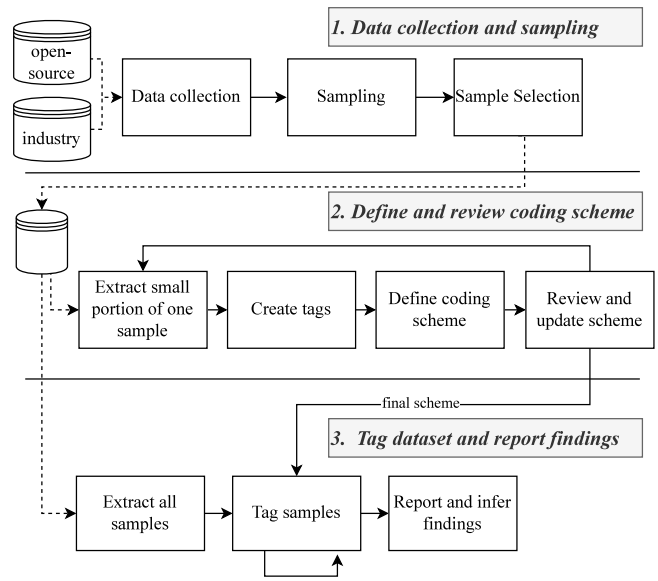


**Fig. 1.** The research approach for RQ1, showing how we (1) collected and sampled the data, (2) defined and reviewed the coding scheme and (3) tagged the data.

This led to 16 candidate OSS projects. Due to our collaboration with Mendix, we knew that the ITSs within the respective seven projects were used by the teams as backlog for the sprints. Therefore, it was unnecessary to perform this selection for the proprietary projects.

*Sampling.* Since the projects varied in size (from 57 to 5,750 issues), we selected a sample from each project. We defined an initial sample size of 100 items, as a sample selection based on a time period would result in a large variation in sample sizes (the projects vary greatly in the item update frequency). The same phenomenon holds for sampling an equal proportion of items per project, since the size of projects differs considerably.

We aimed to select a sample that represents well the backlog items the development teams worked upon in a given time period. Therefore, we randomly selected one issue and included the 99 subsequent issues (not counting bugs) in order of creation time, excluding the last 100 issues from the random selection phase. In other words, considering the dataset as a set of sliding windows of size 100 (and sliding interval of 1), we randomly selected one sliding window per project.

Links between issues provide additional context about how various backlog items relate and depend on one another [25]. Hence, we include this information when extracting samples of the projects. We considered each issue as part of a *cluster* of linked issues. After selecting a sample $S$ of 100 issues, we added all issues that were directly and transitively linked to an issue in $S$, excluding those created after the last issue in $S$.

*Sample selection.* Before the different OSS projects could be used, the two authors independently evaluated the first 12 issues of each sample in order to verify if the initial selection contained a sufficiently high volume of relevant requirements information and were not merely exploited for bug reports and task lists. Each of the 12 issues per project was classified as to whether it contained information related to requirements. After independent tagging, we reviewed the classification and discussed any differences. Finally, the projects with over 50% (of the 12 issues) marked as requirements-relevant were shortlisted. This resulted in 7 OSS projects.

*Selected projects.* Table 1 shows the different projects included in this study, indicating the number of items in the original dataset, the sample size and the labels used. Each backlog item consists of a label, which is specified by one of the team members and should reflect the content of

**Table 1**
The projects selected in this study, showing the total size, sample size, number of items with requirements labels (*E*: Epic, *F*: Feature, *US*: User Story, *SU*: Suggestion), number of task-labeled issues (*T*: Task, *ST*: Sub-task, *TT*: Technical task, *ST*: Support ticket), and other issues.

| Project | Size | | Req-labeled | | | | Task-labeled | | | | Other |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | Total | Sample | E | F | US | SU | T | ST | TT | ST | |
| Control | 738 | 120 | 15 | 0 | 90 | 0 | 0 | 14 | 0 | 1 | 0 |
| Service | 173 | 100 | 6 | 0 | 57 | 0 | 37 | 0 | 0 | 0 | 0 |
| Store | 634 | 109 | 12 | 0 | 69 | 0 | 0 | 0 | 0 | 21 | 7 |
| Company | 29 | 29 | 0 | 0 | 29 | 0 | 0 | 0 | 0 | 0 | 0 |
| Portfolio | 97 | 97 | 4 | 0 | 84 | 0 | 0 | 9 | 0 | 0 | 0 |
| Data | 57 | 57 | 8 | 0 | 27 | 0 | 20 | 2 | 0 | 0 | 0 |
| Learn | 994 | 143 | 15 | 0 | 116 | 0 | 0 | 5 | 0 | 7 | 0 |
| Cost Management (Cost Man.) | 2,038 | 179 | 15 | 8 | 99 | 0 | 28 | 29 | 0 | 0 | 0 |
| Jira Performance Testing Tools (Jira Testing) | 777 | 105 | 2 | 0 | 26 | 57 | 12 | 8 | 0 | 0 | 0 |
| Lyrasis Dura Cloud (Lyrasis) | 1,125 | 113 | 0 | 0 | 105 | 0 | 7 | 0 | 0 | 0 | 1 |
| Network Observability (Network) | 137 | 102 | 2 | 0 | 99 | 0 | 1 | 0 | 0 | 0 | 0 |
| OpenShift UX Product Design (OS UX Design) | 369 | 130 | 3 | 0 | 113 | 0 | 3 | 11 | 0 | 0 | 0 |
| Qt Design Studio (Qt Design) | 4,983 | 180 | 5 | 0 | 51 | 6 | 11 | 21 | 86 | 0 | 0 |
| Red Hat Developer Website (RH Dev.) | 5,750 | 172 | 21 | 0 | 151 | 0 | 0 | 0 | 0 | 0 | 0 |
| Total | 17,901 | 1,636 | 108 | 8 | 1,116 | 63 | 119 | 99 | 86 | 29 | 8 |

the item. In addition to the label, we examine the description, summary (i.e., title) and cluster to which each item belongs.

#### 3.1.2. Coding scheme construction

We constructed the coding scheme through two cycles. In total, three taggers were involved. The primary tagger (the first author) participated persistently across both cycles. A secondary tagger (the second author) contributed predominantly throughout the initial cycle and engaged in discussions during the succeeding cycle. The third tagger (an external researcher) was involved exclusively in the second cycle.

*Cycle 1.* The initial coding scheme was constructed through the analysis of two of the 14 project samples; one OSS (QT Design Studio) and one proprietary (Portfolio). We defined the coding scheme iteratively and performed the tagging using the software NVivo. In each iteration, the first (main) tagger examined a small additional section of data to identify a variety of information in the backlog items, focusing specifically on content and writing patterns. After creating the codes, the first tagger grouped related codes or adjusted codes to construct a coding scheme. When the first tagger was unsettled about certain scenarios, the first and second taggers discussed the situation, adjusting the scheme accordingly. The rest of the process was repeated each time adding new data to adjust the scheme until the scheme no longer required modifications.

After the two projects were fully tagged based on the constructed scheme, the second tagger checked the scheme and the tagged dataset, resulting in some minor final adjustments, leading to the coding scheme that is available in our online appendix [26] and that is briefly explained in Section 3.1.3.

The first tagger applied the scheme to 50% (6) of the remaining projects. The second tagger independently tagged a random set of items (20%) of each project. We subsequently compared the tagged items and discussed conflicts. We consider tagging difference as a conflict when a specific text is tagged with a different granularity level or requirement type. This resulted in an agreement rate of 65%, after which we made a few minor adjustments to six of the eight tagged projects.

Next, the first tagger tagged the remainder of the projects, and the second tagger independently tagged 20% of that sample. After comparing the tags, we found an agreement of 71%, which is an improvement over the previous round. Based on these minor mismatches, we adjusted the tagging. This tagged dataset was used in our previous research [7].

*Cycle 2.* An additional third tagger was recruited in order to increase the tagging reliability. This person independently annotated the 14 projects in accordance with the guidelines established during Cycle 1. In parallel, the main tagger conducted a further review of the previously annotated data. The comparison and discussion of the tagged data between the two coders proceeded iteratively. After the third tagger annotated a segment of the data, the data was compared, followed by a discussion. As a result of these discussions, the guidelines were further adjusted. We assess agreement reliability on the final four projects tagged in this cycle, which led to no further major modifications to the guidelines.

To compare the tagging between the two taggers, we calculate two different Cohen's Kappa scores: (1) whether an item contains a requirement or not, (2) whether the tagged requirements assigned the same type of tags. The first Cohen's Kappa is equal to 0.890; the second Cohen's Kappa is 0.857. Both indicate *almost perfect agreement* between the taggers [27].

#### 3.1.3. Coding scheme

The coding scheme (Table 2) distinguishes two characteristics on which a requirement can be classified: (a) the requirement *type* and (b) the *granularity* level.

The requirement type indicates whether it has a functional or non-functional nature. For functional requirements, we have defined two possible subcategories: (a) user-oriented (the user directly experiences the added functionality), and (b) system-oriented (the added functionality is not directly experienced by the user but is necessary for the system to operate). The granularity level denotes the level of refinement of the requirement, where we distinguish between low-level (e.g., acceptance criteria), medium-level (e.g., user stories), and high-level requirements (e.g., epics). Recognizing the importance of the reason for the requirement, we also tag whether there exists a *motivation* for the requirements in that backlog item. The complete tagging guidelines are available in our online appendix [26].

### 3.2. Results

In this section, we present the findings of our content analysis for RQ1.1–RQ1.3. We address RQ1.1–RQ1.3 in three sections, each of which ends with a reflection on the difference between proprietary and OSS projects wherein we split the projects population between proprietary and OSS projects. Given the small sample size (7 projects per group), we do not run statistical significance tests as their reliability would be low. The raw data are available in our online appendix [26].

#### 3.2.1. Do practitioners choose accurate backlog item labels (RQ1.1)?

RQ1.1 examines whether practitioners' labeling accurately reflects the content of the backlog items and to identify any inconsistencies. We expect backlog items of types 'Epic', 'Feature', 'Story' and 'Suggestion' to contain requirements information, while items with types 'Task', 'Technical task', 'Sub-task' and 'Support-tickets' not to. We exclude 8

**Table 2**
Overview of our coding scheme.

| Characteristic | Category | Abbr. | Description |
|---|---|---|---|
| Requirement type | User-oriented functional | UO | Functionality directly experienced by the user. |
| | System-oriented functional | SO | Functionality that the system will implement but that is not directly experienced by the user. |
| | Non-functional | NF | Requirement that constrains or sets some quality attributes upon functional requirements [28]. |
| Granularity level | Low | L | Requirement that is directly verifiable (e.g., acceptance criterion). |
| | Medium | M | Requirement that refers to one specific functional or non-functional aspect of the system (e.g., user story). |
| | High | H | Requirement that encompasses multiple aspects or functionalities of the systems (e.g., epic or theme). |

issues with a rarely occurring type that are hard to relate to requirements or tasks. Our hypothesis H1 is that the first kind of backlog items will have a higher frequency of requirements-related tags.

**Table 3**
Ratio of task-labeled and requirements-labeled items that include at least one requirement according to our tagging.

| Project (proprietary) | Task | Req | Project (OSS) | Task | Req |
|---|---|---|---|---|---|
| Control | 0.13 | 0.60 | Cost Man. | 0.25 | 0.43 |
| Service | 0.35 | 0.94 | Jira Testing | 0.15 | 0.55 |
| Store | 0 | 0.30 | Lyrasis | 0.14 | 0.76 |
| Company | 0 | 0.55 | Network | 0 | 0.43 |
| Portfolio | 0.22 | 0.50 | OS UX Design | 0.71 | 0.53 |
| Data | 0.64 | 0.97 | Qt Design | 0.21 | 0.47 |
| Learn | 0.25 | 0.44 | RH Dev. | 0 | 0.42 |
| Macro-avg | 0.23 | 0.61 | | 0.21 | 0.51 |
| Std-dev | 0.22 | 0.25 | | 0.24 | 0.12 |
| Macro-avg (all) | 0.22 | 0.56 | | | |
| Std-dev (all) | 0.22 | 0.20 | | | |

Table 3 shows, for each project, the ratio of backlog items that (i) are labeled as tasks and include at least one requirement, and that (ii) are labeled as requirements and include at least one requirement. The columns on the left focus on proprietary projects, the ones on the right on OSS projects.

To confirm H1 statistically, given the small sample size ($n = 14$), we choose a robust non-parametric test: Mann–Whitney's U, verifying if the ratio of requirements in the requirements-labeled items is greater than that in the task-labeled items (H1), or if they can be considered equal (H0). The Mann–Whitney U test results in a test statistic of 173 with $p = 0.0001$. At a significance level of $\alpha = 0.05$, we can reject the claim H0 that the two rates are equal. The effect size is *large*, as $d_{cohen} = 1.716$.

To provide more detailed insights on RQ1.1, Fig. 2 differentiates requirements based on their granularity. The *general* boxplots represent the percentages from Table 3 for those items with requirements in task-related items versus in requirement-related items. The *detailed* view shows the percentage of items with requirements of *medium* or *high* granularity level labeled as task-related versus requirement-related. This view does not include the items that contain only requirements of a low granularity level, as these can be considered closer to work items.

As can be observed, there is only a small difference between the two boxplots (general vs. detailed). The median ratio of task-labeled items containing a requirement in the detailed view is 0.17 ($\sigma = 0.18$) compared to the general view with an median of 0.18 ($\sigma = 0.22$). The ratio of requirement-labeled items with a requirement shows a similar small difference; detailed: 0.49 ($\sigma = 0.20$), general: 0.52 ($\sigma = 0.20$).

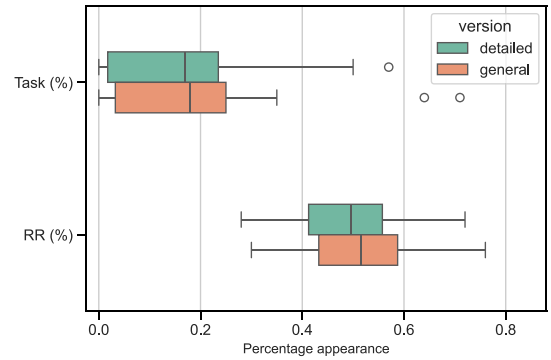


**Fig. 2.** Percentage of requirements in task-labeled items vs. requirements in requirements-labeled items (RQ1.1), distinguishing between their granularity.

When comparing proprietary and OSS projects, task-labeled items exhibit similar results: for proprietary projects $\overline{x} = 0.23$ ($\sigma = 0.22$), and for OSS projects $\overline{x} = 0.21$ ($\sigma = 0.24$). Similarly, when comparing the items labeled as requirements, the proprietary projects yield an average $\overline{x} = 0.61$ ($\sigma = 0.25$) while the OSS projects result in $\overline{x} = 0.51$ ($\sigma = 0.12$). The results indicate a slight difference, with the proprietary projects having on average more items correctly labeled as requirements in comparison to OSS projects, although more projects are necessary to draw general conclusions.

### 3.2.2. RQ1.2: What requirements documentation and refinement patterns can be found in backlog items?

RQ1.2 investigates what type of requirements are frequently present in backlog items, their degree of granularity, and whether the requirements in backlog items are uniquely identifiable [13].

*Prevalent categories.* Fig. 3 shows, for each combination of type and granularity (see Table 2), the occurrence across the 14 projects, distinguishing between proprietary and OSS projects.

When examining the medians per combination, medium-level user-oriented (M-UO) requirements are prevalent in all projects with a median of 0.314 ($\overline{x} = 0.321$, $\sigma = 0.150$). The second more frequent ones are low-level user-oriented (L-UO) requirements with a median of 0.206 ($\overline{x} = 0.266$, $\sigma = 0.188$). The relatively high standard deviation (compared to the $\overline{x}$) and the gap between the median and $\overline{x}$ show that L-UO requirements also exhibit the largest fluctuations across projects.

Fig. 3 shows that certain combinations exhibit a large difference between the OSS and proprietary projects. In particular, for L-UO requirements, the large variation for proprietary projects contrasts with a low one for OSS projects. The results also show a higher percentage of medium-level requirements in OSS projects than in proprietary projects. For high-level requirements and L-NF requirements, only a slight variation exists between project types.

*Multiple requirements.* RQ1.2 also examines whether the requirements in backlog items are uniquely identifiable [13]. We explore the ratio of backlog items with requirement-related information having more than one requirement. Then, we examine which combinations of tags (type and granularity level) are prevalent when 2+ requirements per item are identified. Finally, we investigate how often these items result from a refinement pattern.

Table 4 shows the percentage of requirements-related backlog items consisting of multiple requirements. The *full* column contains all items that contain more than one tagged text segment (this tagging was presented in our previous work [7]), while the *refined* column compensates for situations where there is overlap in tagging. This may be the case, for example, when a requirement identifies both a functional and a non-functional part of the system. An example from the Cost Management

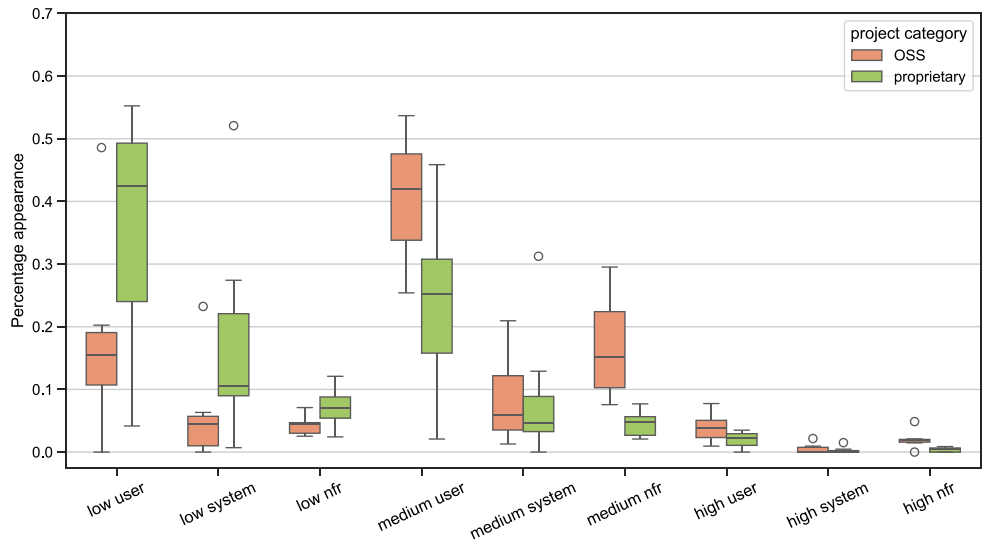


**Fig. 3.** Percentage of occurrence of the type-granularity combinations (RQ1.2).

**Table 4**
Presence of multiple requirements in a single issue.

| Project (proprietary) | Multiple (%) Full | Refined | Project (OSS) | Multiple (%) Full | Refined |
|---|---|---|---|---|---|
| Control | 0.68 | 0.62 | Cost Man. | 0.55 | 0.48 |
| Service | 0.35 | 0.29 | Jira Testing | 0.18 | 0.12 |
| Store | 0.75 | 0.62 | Lyrasis | 0.17 | 0.04 |
| Company | 0.81 | 0.81 | Network | 0.44 | 0.35 |
| Portfolio | 0.83 | 0.74 | OS UX Design | 0.54 | 0.35 |
| Data | 0.67 | 0.46 | Qt Design | 0.37 | 0.35 |
| Learn | 0.57 | 0.47 | RH Dev. | 0.44 | 0.40 |
| Macro-avg | 0.67 | 0.57 | Macro-avg | 0.38 | 0.30 |
| Std-dev | 0.17 | 0.18 | Std-dev | 0.16 | 0.16 |
| Macro-avg (all) | 0.53 | 0.44 | Std-dev (all) | 0.21 | 0.22 |

project: 'As a user, I want to quickly filter my tags based on tag key or value, [...]', which points to the functionality of a filtering option and also to the quality of performing it quickly. Thus, the *full* columns count these cases as two requirements, while the *refined* columns count them as one requirement.

Table 4 shows that the projects in our sample do not only comprise requirements that are uniquely identifiable. The standard deviation indicates large per-project variations. For example, among the items containing requirements in Jira Performance Testing Tools, only 18% (or refined 12%) contain items with multiple requirements, while the project Company has more than 81% items containing multiple requirements.

Table 5 shows the most common combinations of different tags that co-occur in an item, including only those combinations having at least 10 total occurrences. Each (project) column represents the ratio of the occurrence of a particular combination compared to all possible combinations of different tags within a single items in the given project. The *total* column represents the ratio of a particular combination over all possible combinations in the entire dataset. The most frequent combination is having a medium-level requirement refined into 1+ low-level requirements of the same type, which occurs in 12/14 projects. The second most common combination is two medium-level requirements: one non-functional and one functional user-oriented, also occurring in 12 projects. The third is complementary to the first: in several projects, M-SO requirements are refined into L-SO requirements.

Rows 1, 3, 4 and 5 of Table 5 all show combinations that could reflect a *refinement pattern* within a single backlog item. We consider a refinement as one requirement with medium or high granularity level being further detailed by 2+ requirements with lower granularity. This covers two scenarios: a medium-level requirement refined into low-level requirements, and a high-level requirement refined into lower-level requirements.

Table 6 presents the ratio of items that include a refinement over the total number of items with multiple requirements. High-level requirements are hardly refined within the same issue, while medium-level requirements are more often refined in the same issue ($\overline{x} = 0.53$). These ratios vary considerably depending on the project, as indicated by the standard deviation.

The results illustrate substantial variations between proprietary and open-source projects concerning the inclusion of multiple requirements within a single item. Table 4 compares the two types of projects in terms of the presence of multiple requirements. On average, the proprietary projects hold more items with multiple requirements ($\overline{x} = 0.57$, $\sigma = 0.18$) than the OSS projects ($\overline{x} = 0.30$, $\sigma = 0.16$). The standard deviations likewise show larger fluctuations in the proprietary projects. Moreover, the refinement of high-level requirements into medium-level requirements is most common in the OSS projects, while the refinement of medium-level requirements is more frequent in proprietary projects.

### 3.2.3. Are requirements complemented by a motivation (RQ1.3)?

RQ1.3 examines whether a backlog item containing requirements includes an associated motivation. Table 7 shows the percentage of backlog items where we identified at least one requirement that contain at least one justification. The *all* column considers all items having at least one requirement (regardless of type or granularity level), while the *detailed* column includes only those items possessing medium- or high-level requirements. The overall macro-average shows that many requirements do not have an associated motivation in the backlog. The more granular view provided by the *detailed* column does not reveal significant differences regarding this observation.

The standard deviation of 0.17 (0.18 for the *detailed* view) also indicates that the percentages fluctuate only slightly across the projects. In 8/14 projects, less than 50% of the requirements-containing items have an associated justification. The other six projects have less than 65% of their backlog items with motivations.

Table 7 compares the type of projects in terms of the presence of motivation. The macro average reveals only a small difference, with OSS projects having slightly more frequent motivations in their backlog items.

**Table 5**
Most frequent combinations of different tags in the same issue, showing both the total and the per-project counts.

| Combinations | Total | Control | Service | Store | Company | Portfolio | Data | Learn | Cost Man. | Jira Testing | Lyrasis | Network | OS UX Design | QT Design | RH Dev. |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| low user, medium user | 0.026 | 0.41 | 0.44 | 0.22 | 0 | 0.45 | 0.47 | 0.38 | 0.08 | 0.22 | 0 | 0.37 | 0.13 | 0.20 | 0.50 |
| medium nfr, medium user | 0.010 | 0.07 | 0.16 | 0.06 | 0 | 0 | 0.06 | 0.03 | 0.08 | 0.33 | 0.64 | 0.05 | 0.33 | 0.10 | 0.12 |
| low system, medium system | 0.005 | 0.07 | 0 | 0.17 | 0.38 | 0.08 | 0 | 0 | 0.11 | 0 | 0 | 0.05 | 0 | 0 | 0.06 |
| low user, medium user, low system | 0.005 | 0.16 | 0.12 | 0.06 | 0 | 0.08 | 0 | 0.15 | 0.03 | 0 | 0 | 0.05 | 0 | 0 | 0 |
| low user, low nfr, medium user | 0.003 | 0.07 | 0 | 0.06 | 0 | 0.05 | 0.09 | 0 | 0.03 | 0 | 0 | 0 | 0.05 | 0 | 0.03 |
| low user, medium nfr, medium user | 0.002 | 0 | 0 | 0 | 0 | 0 | 0.16 | 0 | 0.03 | 0 | 0 | 0.05 | 0 | 0.05 | 0.06 |

**Table 6**
Presence of refinement patterns in a single issue.

| Project (proprietary) | Refinement (%) High | Refinement (%) Medium | Project (OSS) | Refinement (%) High | Refinement (%) Medium |
|---|---|---|---|---|---|
| Control | 0.03 | 0.83 | Cost Man. | 0.16 | 0.41 |
| Service | 0 | 0.62 | Jira Testing | 0 | 0.50 |
| Store | 0 | 0.53 | Lyrasis | 0.33 | 0 |
| Company | 0 | 0.62 | Network | 0.07 | 0.47 |
| Portfolio | 0 | 0.79 | OS UX Design | 0.16 | 0.28 |
| Data | 0 | 0.59 | Qt Design | 0.16 | 0.05 |
| Learn | 0 | 0.68 | RH Dev. | 0 | 0.59 |
| Macro-avg | 0.0 | 0.67 | Macro-avg | 0.12 | 0.38 |
| Std-dev | 0.01 | 0.11 | Std-dev | 0.12 | 0.19 |
| Macro-avg (all) | 0.06 | 0.53 | Std-dev (all) | 0.10 | 0.21 |

**Table 7**
Backlog items with at least one requirement that also have a motivation.

| Project (proprietary) | Yes (%) All | Yes (%) Detailed | Project (OSS) | Yes (%) All | Yes (%) Detailed |
|---|---|---|---|---|---|
| Control | 0.12 | 0.13 | Cost Man. | 0.60 | 0.64 |
| Service | 0.56 | 0.57 | Jira Testing | 0.62 | 0.62 |
| Store | 0.63 | 0.61 | Lyrasis | 0.41 | 0.42 |
| Company | 0.25 | 0.27 | Network | 0.23 | 0.24 |
| Portfolio | 0.41 | 0.42 | OS UX Design | 0.50 | 0.50 |
| Data | 0.56 | 0.60 | Qt Design | 0.30 | 0.31 |
| Learn | 0.22 | 0.21 | RH Dev. | 0.26 | 0.27 |
| Macro-avg | 0.39 | 0.40 | Macro-avg | 0.42 | 0.43 |
| Std-dev | 0.20 | 0.20 | Std-dev | 0.16 | 0.17 |
| Macro-avg (all) | 0.40 | 0.42 | Std-dev (all) | 0.17 | 0.18 |

### 3.3. Discussion

We address each of the research questions on the basis of the findings reported in Section 3.2. We highlight the most remarkable results and provide additional probable explanations.

*RQ1.1: To what extent do the backlog item labels chosen by practitioners reflect the requirements expressed in the items?* The hypothesis H1 tested whether the percentage of requirements in requirements-labeled items is higher than in task-related items. The results reported in Section 3.2.1 confirm the hypothesis, indicating it is more likely to find requirements in items labeled as such than in items labeled as tasks. The average percentages in Table 3 are more surprising. On average, 22% of the items labeled as tasks do in fact contain requirements. In addition, on average, more than 40% of the items labeled as requirements contain no requirements. The results do not change when excluding lower-level requirements (e.g., acceptance criteria), which can be considered as more closely related to the tasks. These results show that practitioners do inconsistently label the items; therefore, in order to locate requirements within backlog items, it is not sufficient to simply display the issues that are labeled as requirement (here: epics, features, user stories, and suggestions).

There is a notable difference between the OSS projects and the proprietary dataset in labeling the backlog items, with the OSS projects showing more inconsistencies when considering the items labeled as requirements-related (see Table 3). This lack of consistency in OSS projects may be due to the limited oversight or the varying experience level of contributors.

**Finding 1.1.** On average, less than 60% of the items labeled as requirements contained one or more requirements, while 22% of the items not labeled as requirements in fact contained requirements.

*RQ1.2: What requirements documentation and refinement patterns can be found in backlog items?* Fig. 3 visualizes the occurrence of different types of requirements with specific granularity levels in backlog items. There is a high fluctuation between certain categories, especially for low- and medium-level user-oriented functional requirements. This reflects the different usage patterns of teams managing their product or sprint backlogs. For example, the combination low-level functional requirements ('low user' in Fig. 3) for proprietary projects shows high variance; this happens because some teams include acceptance criteria in the same issue where a user story is written, while others do not specify them, or store them in a different environment.

For each project, Table 4 shows how many of the items with at least one requirement do contain multiple requirements. Many of these backlog items contain multiple requirements, making them no longer uniquely identifiable. Nevertheless, this is strongly dependent on the project.

Table 5 indicates which different categories of requirements often occur together and Table 6 presents a detailed analysis of the number of items exhibiting multiple requirements due to a refinement pattern. Table 6 demonstrates that a significant proportion of items possessing multiple requirements are the result of a refinement pattern. Considering the cases in Table 5, we find that the most common combination is M-UO requirements coupled with L-UO requirements. This is a generalization of the refinement pattern where a user story is refined into acceptance criteria. In addition, the results in Table 5 show that non-functional requirements often co-occur with functional requirements (which is not considered a refinement pattern). This could be interpreted in terms of the conversion of non-functional requirements into functional requirements that are closer to system design [29], which can be expressed within a single extensive requirement.

**Finding 1.2.** A common pattern in items with multiple requirements is the combination of medium and low-level user-oriented functional requirements, such as a user story refined into acceptance criteria.

Although Table 6 indicates that a large proportion of items with multiple requirements are the result of a refinement pattern, there is still a significant proportion of items containing multiple requirements that are not the result of refinement. Moreover, although the choice is intuitive, the refinement pattern of high to medium requirements in the same issue should be avoided because these requirements should be identifiable. Consequently, 47% of items with multiple requirements present challenges due to the presence of multiple medium or high-level requirements that ought to be uniquely identifiable.

**Finding 1.3.** More than 40% of the items with multiple requirements are not the result of a refinement pattern, presenting challenges for unique identification.

Finally, when comparing OSS and proprietary projects, we observe that proprietary projects in our sample show large variations between teams in their structuring of the project backlog (e.g., the type of included requirements), and more often document multiple requirements in a single backlog item (Table 4).

*RQ1.3: To what extent are requirements complemented by a motivation for their existence?.* More than half of the backlog items that contain at least one requirement did not include any justifications for those requirements. This shows lack of recognition of the importance [14] and recommendations [13] for expressing the 'why' behind requirements. This may be due to agile software development practices, where requirements are formulated in a concise manner, as they are intended to support and foster the conversation within the team [30], rather than acting as a precise and complete specification. Also, Table 7 considers the presence of justifications for all categories of requirements compared to items containing only medium or high-level requirements. This distinction does not change the resulting observation.

**Finding 1.4.** The justification of requirements is often absent in backlog items.

## 4. Part 2: Identifying and Classifying Requirements in ITSs with LLMs

This section addresses RQ2 regarding locating and classifying requirements in backlog items, which is motivated by the results from Part 1; in particular, by (i) the discrepancies between the issue labeling and the presence of requirements (Finding 1.1) and (ii) the presence of multiple requirements within a single backlog item (Finding 1.3). Our experiments explore two distinct classification tasks:

Task1 Requirements Identification: determining whether a segment of a backlog item comprises a requirement.
Task2 Requirement Type Classification: assigning a type to a segment that was identified as a requirement in Task1.

In Section 4.1, we discuss the dataset alongside the pre-processing steps. Then, we describe the LLMs we selected (Section 4.2) and the metrics chosen to evaluate these different models (Section 4.3). Finally, in Section 4.4 and Section 4.5, we present and discuss the results.

### 4.1. Dataset

The dataset employed in these experiments is derived from the annotated data of the content analysis. Table 1 in Section 3.1 presents preliminary descriptive statistics for this dataset.

#### 4.1.1. Pre-processing procedure

The pre-processing procedure follows the steps presented in Fig. 4, which primarily consist of segmentation and dividing the data between validation and training splits.

*Segmentation.* Based on the initial observations in Part 1, we discovered that requirements are not exclusively specified in one sentence, but can span across several sentences. Therefore, we wrote a Python script that divides each item into segments based on whether it matches one of the following patterns (see the appendix for an example of this segmentation process visualized in Figure A [26]):

1. Paragraph: we separate individual paragraphs by the presence of a at least two newline commands between them.[3]

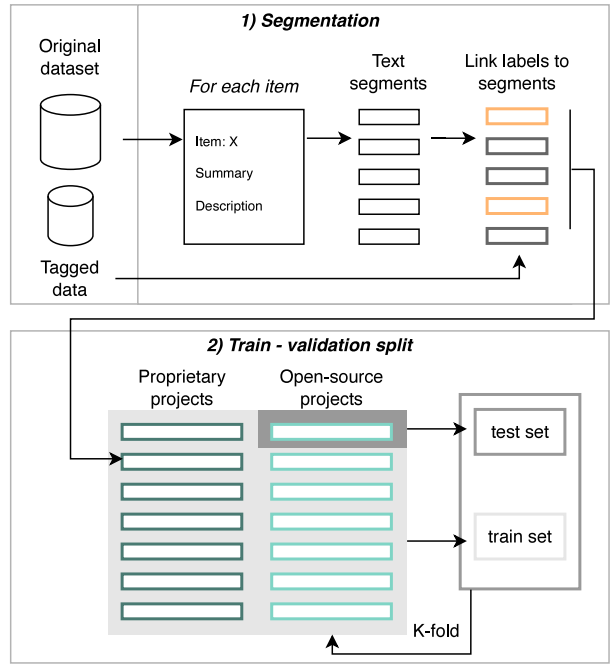


**Fig. 4.** Pre-processing procedure applied to RQ2.

2. Enumerations and bullet lists: we split each individual bullet point (e.g., '*' are most common in our exported data from Jira) or enumeration item ('1.', '2.', etc.) into separate segments. We split only enumerations and bullet lists at the first level, discarding sub-enumerations or sub-bullet lists, as we found that these generally represent a clarification that relates to the parent bullet item. Furthermore, if a bullet point does not include a verb, it is likely that the enumeration is directly connected to the previous level and, therefore, we do not split it.

After segmenting the data, the Python script associates each extracted text segments with the corresponding labels that we assigned in Part 1 of this research, indicating whether that segment contains a requirement (label for Task1) and with what characteristics (label for Task2).

In Part 1 (Section 3), we have labeled particular requirements with two codes (for the characteristic: requirement type). This occurs when the requirement describes both a functional and non-functional aspect of the system. In these cases, a requirement has multiple labels. Since these instances account for less than 4% of the total requirements, we opted to exclude these cases from the experiments of Task2 to avoid unnecessary complexity in the reporting of the results.

*Train/validation splits.* After segmentation, each project set consists of a number of labeled segments. To evaluate the performance of the various language models, we use group k-fold cross-validation (a.k.a. project-fold [31]). K-fold cross-validation is widely used to ensure robust evaluation, where a fold between 5 and 10 is recommended to balance bias, variance and computational cost for the error estimation [32]. To avoid data leakage, we perform project-fold cross-validation. Due to confidentiality reasons, we exclusively use OSS data to assess the performance of the decoder-only models and encoder-only models; proprietary data is used solely for training the encoder-only models. Applying project-fold for the encoder-only models entails taking 1 OSS project as the test set (the left-out project) and 6 OSS projects to train the classifier plus the 7 proprietary projects, repeating this same procedure 7 times by leaving out always one OSS project. Finally, we report the average and standard deviation in predicting the

[3] Eight segments, even after paragraph segmentation, still included over >2,000 tokens. Those were further split on a single newline.

left-out project. The same 7 OSS projects are also applied to evaluate decoder-only models.

We do not perform hyper-parameter tuning on the models, as our aim is that of assessing the overall performance without running into risks of overfitting. Thus, we did not create an additional test set; we use the results obtained from different splits to evaluate performance.

#### 4.1.2. Final datasets

Table 8 shows additional information on the final datasets. The original dataset comprises a prominent imbalance for both tasks (requirement vs. no requirement, and user vs. system vs. non-functional). To examine the potential impact on the performance of encoder-only models, we derived rebalanced versions of the dataset via random undersampling (details below). The lower part of Table 8 shows details on the balanced datasets.

**Table 8**
Descriptive statistics for our dataset. Avg/std-dev %: average/standard deviation percentage of respective class across projects.

| | # segments | Avg % | Std-dev % |
|---|---|---|---|
| Original | | | |
| No requirement | 8478 | 74.9 | 15.6 |
| Requirement | 2106 | 25.1 | 15.6 |
| User | 1375 | 64.7 | 20.4 |
| System | 352 | 20.6 | 21.2 |
| Non-functional | 295 | 14.6 | 7.2 |
| Balanced | | | |
| No requirement | 2990 | 58.2 | 4.7 |
| Requirement | 2106 | 41.8 | 4.7 |
| User | 457 | 42.1 | 9.3 |
| System | 308 | 28.8 | 17.5 |
| Non-functional | 291 | 29.1 | 12.3 |

We re-sampled the data using two approaches, one for each task. To construct the balanced set for Task1 (first two rows of 'Balanced' in Table 8), we set a minimum threshold for the minority class (40% of the dataset), with a uniform representation across the projects. This threshold leads to an imbalance ratio (IR) of 1.5, which is commonly used (see, for example, [33]). For Task2, we follow a slightly different approach due to the limited amount of data with high variation across the different classes. With the goal of maintaining an *overall* average IR of 1.5 (over all project), we determined for each project if the minority class was significantly low: more than 2 times lower than the majority class. In those cases, we could set the majority classes to the average of the minority class across all projects (as otherwise the reduction in the majority classes would be too severe, e.g., if the minority class is equal to 1); alternatively, we maintained an IR of 2 for the class. This approach resulted in an overall IR of approximately 1.5.

We used the balanced data exclusively for training the respective models. When evaluating the models, we use the original data as test sets in all cases (instead of the balanced set), regardless of the data used for training.

### 4.2. Models

As per RQ2.1 and RQ2.2, we aim to evaluate and compare the performance of encoder-only models against decoder-only models. Hence, we have selected well-recognized models to represent each category. The training and prediction of the models are performed using Python 3.9.

#### 4.2.1. Encoder-only

To explore the performance of encoder-only models in executing the two tasks, we experiment with two state-of-the-art encoder-only models: BERT and RoBERTa. BERT, short for Bidirectional Encoder Representations from Transformers, was first introduced by Google [34] and has been adopted extensively across a wide range of natural language processing (NLP) activities. Facebook AI [35] proposed a refined version of BERT, named a Robustly Optimized BERT Pretraining Approach, RoBERTa, designed to improve BERT's performance.

Our experiment studies the performance of BERT (base, cased) and of RoBERTa (base), including fine-tune adjustments to adapt the models to the two designated downstream tasks. Although we did not perform hyperparameter tuning, we did apply widely accepted and previously tested hyperparameter values for BERT and RoBERTa, such as a batch size of 16 and a learning rate of 1e–5 [34–36]. Additionally, our architecture incorporates a dropout layer after the general architectures to mitigate overfitting during the training process (we remove this layer when predicting the labels). To transform the output to the appropriate output range, we use a ReLu activation function [37], one of the most widely adopted activation functions (whereas variants of ReLu struggle to show sufficient performance gains [38]). To retrieve the best model across all epochs, we apply an early stopping mechanism with a tolerance of 4 epochs and a maximum of 10 epochs.[4] We built the encoder-models using Pytorch and retrieved the pre-trained models from HuggingFace. As for both BERT and RoBERTa, we have two versions in our experiment: one trained on the original and one trained on the balanced dataset. The names 'RoBERTa_1' and 'BERT_1' models denote the encoder-only models trained on the original dataset; 'RoBERTa_2' and 'BERT_2' models are trained on the balanced dataset.

#### 4.2.2. Decoder-only

For the decoder-only models, we select three widely recognized models: ChatGPT, Mistral 7B and Llama 3.

To implement Mistral 7B and Llama 3, we use the Python library of GPT4All,[5] an open-source software that enables LLMs to be run locally. Due to the token limits, we prompted the model per item, asking the LLM to predict each segment in that item. For items that still exceed the token limit, we use multiple prompts. In addition, to maximize determinism, we set the temperature to 0.

In addition to Mistral 7B and Llama 3, we chose ChatGPT, as this model is currently receiving the most attention. We use the GUI version of GPT 4 and provide the model with an Excel file containing a text segment for each row. The appendix includes the prompt for this experiment [26].

After an initial exploratory phase, we discovered that ChatGPT was unable to produce predictions for all segments at once. Therefore, we retrieved the predictions by project and, if the project exceeded the limit of 500 segments, we split it into segments of 500 (or less).

*In-context learning.* To better assess the potential of decoder-only models, we also measure their performance when applying in-context learning (ICL), where the model learns from a set of demonstration examples given in a prompt. As opposed to retraining or fine-tuning, model parameters are not updated. ICL expects the LLM to learn from the examples without adding substantial computational effort [39]. To compare decoder-only models with and without in-context learning, we have selected ChatGPT as a representative model.

The selection of adequate examples is essential to ICL. We adopt a modified version of the state-of-the-art unsupervised learning method KATE [40]. This method selects demonstration examples using the top k nearest neighbors based on the distance between the embedding representations of the entries. In our case, when selecting examples for a batch of a specific project, we consider segments from the other OSS projects.

Given that this approach also employs encoder-only models to create embeddings, this strategy uses both encoder-only models and decoder-only models. Specifically, Liu et al. [40] use RoBERTa-large

[4] This maximum is rarely reached across the k-fold splits for the different models.

[5] https://docs.gpt4all.io/index.html.

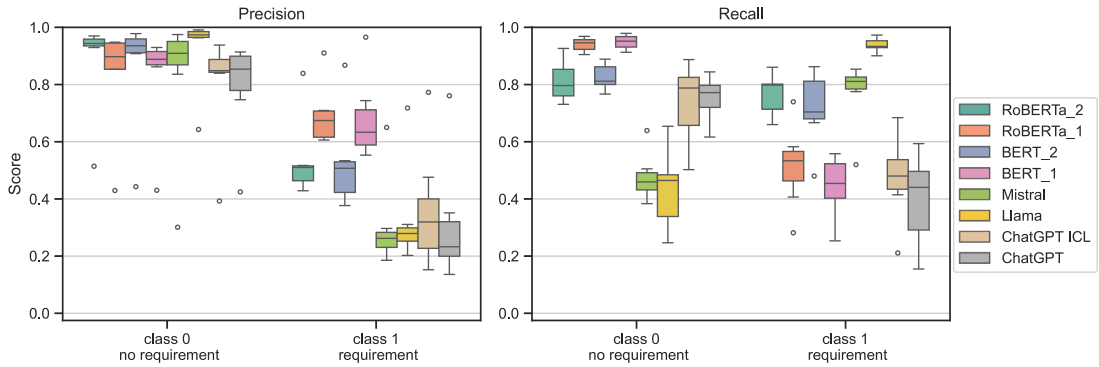


**Fig. 5.** Precision and recall across models for requirements identification.

to create the embeddings and selects Euclidean distance metric to calculate the distances.

Since we use ChatGPT in batches, rather than at the single item level, we adapt the approach. For each segment in a batch file, we select the nearest neighbor segment from the other OSS projects (excluding duplicates).

*Prompt creation.* To compose the prompts, we used the recommendations proposed by White et al. [41] and OpenAI [42]. We focused on the tactics we deemed most appropriate for our experimental setting:

- Persona: we instruct the LLM to act in accordance with a particular persona, which should help consider a particular viewpoint [41,42]. In our case, the persona is that of a requirements engineer.
- Template: including the template pattern in a prompt is expected to help the LLM organize the output in a given structure [41]. In the prompts, we specify the desired output format (a sequence of `<segment number>: <answer>`) for Llama and Mistral, and ask ChatGPT to return an Excel output file.
- Context: in the base prompt (without ICL) we are not providing LLMs with concrete examples. Rather, we provide a general description of each category. For instance, for the classification task, we state that an UO requirement may be a user story, a SO requirement may be back-end related, and a NF requirement may focus on security or usability. For ICL, we provide a demonstration example for each segment in the batch.
- Structure: Structuring the prompt by using separators can be useful to distinguish sections that should be treated differently [42]. For example, when we give the LLMs demonstration examples, we place the examples between three quotation marks.

The final prompts can be found in the appendix in the sub-folders dedicated to decoder-only models [26].

### 4.3. Metrics

To gain a thorough understanding of the models' performance in accurately predicting the respective classes, we use three metrics:

- Recall: for each class, we calculate the percentage of true class instances that are accurately predicted by the model.
- Precision: for each class, we calculate the ratio of predictions for a particular class that accurately belong to that class.
- F1 score: represents the harmonic mean between precision and recall.

We choose to use these metrics to understand the extent to which the models are able to retrieve all requirements (recall) versus how precise the predictions are (precision). Both metrics are valuable, but for the task of identifying requirements, it is marginally more important to locate all requirements as opposed to having limited noise in the final selection of requirements. We additionally selected the F1 score to understand the overall balanced performance of the models.[6]

### 4.4. Quantitative findings

We report on the performance of encoder-only and decoder-only models in (i) identifying requirements (Section 4.4.1) and in (ii) classifying the requirement type (Section 4.4.2).

#### 4.4.1. Requirements identification

The left plot of Fig. 5 shows the precision score across all OSS projects for each model, while distinguishing the scores by class. The raw precision scores are placed in the appendix [26]. Class 0 refers to the segments with no requirements, while Class 1 indicates the segments with requirements.

Most models are able to accurately predict the segments without requirements, while the results for the segments with requirements are not as favorable and they show high variation. None of the decoder-only models (ChatGPT, Mistral, Llama) are able to outperform any of the encoder-only models in terms of precision of class 1. The optimal precision for predicting class 1 is achieved by RoBERTa_1 and BERT_1 when considering the median value.

To test whether the difference in precision score between the models is significant, we first perform the Friedman test. This non-parametric statistical test results in a test statistic of 41.38 with $p < 0.01$ (n = 8), when using a macro average across the classes. Hence, with $\alpha = 0.05$, the precision scores are significantly different. To further explore which pairs of models show statistically significant differences, we perform Nemenyi's post-hoc test [43]. Figure B in the appendix visualizes the p-values by considering the macro average across the classes to compare the different models [26]. With $\alpha = 0.05$, RoBERTa_1 and BERT_1 show a significant difference with Mistral, ChatGPT and ChatGPT with ICL in terms of precision. In addition, RoBERTa_2 differs significantly with ChatGPT ($p = 0.047$).

The right plot of Fig. 5 shows the performance in terms of recall. Compared to precision, there is higher variation between the decoder-only and encoder-only models. The Mistral and Llama models score highest for the requirements class, but the lowest for non-requirements. ChatGPT, the other decoder-only model, more closely aligns with the encoder-only models; nonetheless, ChatGPT's performance (without ICL) is inferior to the encoder-only models, as evidenced by its lowest median recall. The performance of ChatGPT improves with ICL (when observing the median value). It does not, however, surpass the performance of encoder-only models. Another noticeable observation is that the BERT and RoBERTa models, when trained on a balanced dataset,

[6] We selected recall and precision in order to analyze each of these important metrics in isolation, as opposed to studying various weighted versions of the harmonic mean ($F_\beta$).

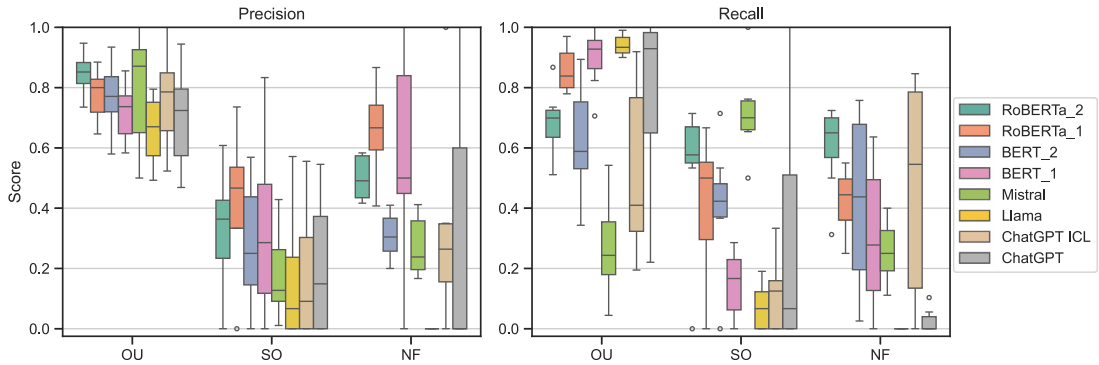


**Fig. 6.** Precision and Recall across models for requirements classification.

ensure a finer balanced trade-off between the two classes in terms of recall.

The Friedman test for the recall scores results in a statistic of 38.43 with $p < 0.01$; with $\alpha = 0.05$, the recall scores are significantly different. The Nemenyi's results shows that, with $\alpha = 0.05$, the encoder-only models trained on the balanced dataset deviate significantly from Mistral and ChatGPT (with and without ICL) in terms of recall. In addition, at $\alpha = 0.10$, RoBERTa_1 differs significantly from ChatGPT (without ICL). The raw Nemenyi's results are presented in the middle heatmap of Figure B in the appendix.

Table 9 presents the F1-scores. In general, the encoder-only models exhibit a higher F1-score than the decoder-only models. The highest F1-score for class 1 is obtained by RoBERTa_2, followed by BERT_2.

**Table 9**
Average F1-score for requirements identification.

| Model (Encode) | F1 score (%) 0 | 1 | Model (Decode) | F1 score (%) 0 | 1 |
|---|---|---|---|---|---|
| BERT_1 | 0.87 | 0.53 | ChatGPT | 0.77 | 0.33 |
| BERT_2 | 0.84 | 0.58 | ChatGPT ICL | 0.75 | 0.38 |
| RoBERTa_1 | 0.88 | 0.57 | Llama | 0.58 | 0.47 |
| RoBERTa_2 | 0.84 | 0.62 | Mistral | 0.59 | 0.40 |
| Macro avg | 0.86 | 0.57 | | 0.67 | 0.40 |
| Std-dev | 0.02 | 0.04 | | 0.10 | 0.06 |

The Friedman test for the F1-scores results in a statistic of 35.04 with a $p < 0.01$; hence, with $\alpha = 0.05$, the F1-scores are significantly different. Figure B in the appendix visualizes the p-values for Nemenyi's post-hoc test for the F1-scores. With $\alpha = 0.05$, both RoBERTa models differ significantly from Mistral and Llama (with $p < 0.001$ for both RoBERTa models vs. Mistral). In addition, both BERT models are significant different from Mistral ($p < 0.05$) and both versions of RoBERTa significantly differ from ChatGPT (using ICL) with $\alpha = 0.10$.

#### 4.4.2. Classifying requirement type

We analyze the performance of the encoder-only versus the decoder-only models for classifying the requirement type: user-oriented functional requirements (UO), system-oriented functional requirements (SO) and non-functional requirements (NF).

Fig. 6 shows the precision score in the left plot across the OSS projects for each of the models and separates the scores by class. There is a marked difference in precision for the various classes. For all models, the precision value for class UO is the highest. For the SO class, the performance of the RoBERTa models is superior in terms of the median value. For the NF class, BERT and RoBERTa have the highest median value. Furthermore, for class SO, nearly every model has at least one project where no instance is correctly predicted; for class NF, this is the case for the BERT_1, ChatGPT and Llama. Noteworthy, Llama cannot correctly predict a single instance for class NF.

The Friedman test for precision results in a test statistic of 27.19 with a $p < 0.01$; with $\alpha = 0.05$, the precision scores are significantly different. When exploring the differences via Nemenyi's post-hoc test (Figure C in the appendix), with $\alpha = 0.05$, the encoder-only models trained on the original dataset and the balanced RoBERTa version are significantly different from Llama. In addition, the RoBERTa_1 significantly differs from ChatGPT (without ICL).

The right plot of Fig. 6 shows the recall scores across the OSS projects, split per class: UO, SO, and NF. The figure reveals substantial variation across models and classes. Most models have a median recall above (or close to) 0.4 for class UO, except for the Mistral model which, instead, performs the best for class SO, followed by both versions of RoBERTa. Similarly, for class NF, both versions of RoBERTa as well as ChatGPT ICL have the highest median recall score. Interestingly, ChatGPT (without ICL) exhibits significant variance between projects for class SO, where it correctly predicts all instances for one project and zero for another. For identifying the NF class, the performance of ChatGPT improves significantly with ICL. In addition, for recall, Llama has a score of near 0 across all projects for the class NF.

The Friedman test for the recall scores results in a statistic of 38.42 with $p < 0.01$. The results of Nemenyi's post-hoc test for recall indicate that both RoBERTa versions are significantly different from the Llama model and ChatGPT (with and without ICL). Figure C in the appendix shows the raw Nemenyi's results.

Table 10 shows the F1-scores of the individual models for predicting the different classes. As with the requirements identification tasks, the encoder-only model shows a superior F1-score compared to the decoder-only models. The models with the highest F1-score are the RoBERTa models.

**Table 10**
Average F1-score for requirements classification: UO, SO, NF.

| Model (Encode) | F1 score (%) UO | SO | NF | Model (Decode) | F1 score (%) UO | SO | NF |
|---|---|---|---|---|---|---|---|
| BERT_1 | 0.79 | 0.18 | 0.34 | ChatGPT | 0.69 | 0.14 | 0.05 |
| BERT_2 | 0.67 | 0.31 | 0.31 | ChatGPT ICL | 0.57 | 0.13 | 0.25 |
| RoBERTa_1 | 0.81 | 0.40 | 0.50 | Llama | 0.77 | 0.09 | 0.0 |
| RoBERTa_2 | 0.75 | 0.39 | 0.54 | Mistral | 0.39 | 0.27 | 0.26 |
| Macro avg | 0.76 | 0.32 | 0.42 | | 0.60 | 0.16 | 0.14 |
| Std-dev | 0.06 | 0.10 | 0.11 | | 0.17 | 0.08 | 0.14 |

The Friedman test for the F-score scores results in a statistic of 36.33 with $p < 0.01$. The results from Nemenyi's post-hoc (Figure C in the appendix) indicate that, with $\alpha = 0.05$, both RoBERTa versions are significantly different from the four decoder-only models.

### 4.5. Discussion

We discuss the performance of the models for automatically locating and classifying requirements in backlog items, then we put forward practical implications.

#### 4.5.1. Quantitative findings

*Requirements identification.* As indicated in Section 4.3, we prioritize recall, as long as precision is not excessively affected. Hence, although Llama is the best model in terms of recall for class 1, its recall for class 0 is the second lowest of all models and its precision score for class 1 is one of the lowest (Fig. 5). This performance is sub-optimal, as this implies that Llama would assume most segments are requirements, with many of these being noise (false positives).

The balanced encoder-only models have a finer trade-off between classes. Even though the BERT and RoBERTa models trained on the original dataset show a higher recall in predicting class 0, the main class of interest is class 1. These results of BERT_2 and RoBERTa_2 reveal that the balancing of the data substantially influences performance results for predicting whether a text segment is a requirement or not. This leads to our first finding:

**Finding 2.1.** Encoder-only models trained on a balanced dataset outperform encoder-only models trained on unbalanced dataset and significantly outperform the decoder-only models in terms of recall.

The Llama and Mistral models show low precision when predicting class 1. This can be explained by the high recall score of class 1, where segments are 'over-predicted' as requirement. Overall, encoder-only models perform better for precision, as none of the decoder-only models are able to outperform the encoder-only models for class 1. As indicated in the previous section, this difference is largest between the models trained on the original dataset and the four decoder-only models. This higher precision for the encoder-only models trained on the raw dataset can be explained by the fact that the dominance of the majority class in the training set is causing the models to be more careful in predicting the minority class.

**Finding 2.2.** Encoder-only models show higher precision, particularly in predicting whether a segment is correctly a requirement, and is most significant when trained on the raw dataset, prior to rebalancing.

The F1-scores in Table 9 highlight how the encoder-only models have on average a higher performance for both classes. This difference is statistically significant, except for the BERT models. The higher F1-score can be explained by the fact that the encoder-only models (especially those trained on the balanced dataset) have a more harmonized performance between the classes, and between recall and precision.

**Finding 2.3.** Encoder-only models have on average a higher F1-score for predicting both whether a segment is a requirement or not.

When comparing the performance difference between ChatGPT with or without ICL, performance appears to have improved. In particular, precision and recall improved when predicting instances containing a requirement; a significant improvement, however, could not be found.

The results of Task1 witness the importance of fine-tuning a language model to the task. For example, despite ChatGPT being trained on a significantly larger dataset compared to BERT and RoBERTa, it cannot overcome the benefits from fine-tuning.

*Classifying requirement type.* For this task, all models struggle with accurately predicting the SO and NF classes. Mistral achieves a high recall metrics for the SO class, but at the expense of precision. The challenge in predicting the NF class may be due to implicit formulation of non-functional requirements as certain NF requirements in backlog items do not specifically state the non-functional part.

In addition, performance may be affected by the limited number of data points compared to Task1. Nevertheless, even with the more balanced dataset, the BERT model faces challenges in predicting the SO and NF classes. This leads to:

**Finding 2.4.** Both encoder-only and decoder-only models exhibits good predictive performance for requirements type UO, whereas they face challenges in achieving accurate predictions for SO and NF classes.

Llama is remarkably inaccurate, as it can hardly predict correctly any of the instances of class NF. We further verified whether this could be caused by the order in which each class appears in the prompt, since functional requirements are listed and explained first and the non-functional class is mentioned only last. Thus, we ran the experiment with Llama a second time using a different order, yet this did not alter the prediction; i.e., Llama is simply unable to identify non-functional requirements in the current set-up.

As shown in Section 4.4, the performance of the models between Task1 and Task2 is noticeably divergent. This could be accounted for by: (1) the higher complexity of Task2 (e.g., prediction of multiple classes instead of binary classification), (2) the models for Task2 are trained on less data. Likewise, this partly explains the weak impact that data balancing has on the performance of the encoder-only models for Task2. Still, the RoBERTa models outperforms the other decoder-only models, leading to:

**Finding 2.5.** Encoder-only models (specifically RoBERTa) outperforms the decoder-only models in terms of precision, recall and F1-score for classifying the type of requirement.

#### 4.5.2. Practical implications

While the performance of a model is crucial, other characteristics should be considered when integrating a model into a tool to be used in practice.

A disadvantage of Llama and Mistral is the additional manual output checking that they require. Llama shows inconsistencies in following the template specified by the prompt: although we prompted the models to enter their answers into a dictionary format, in some situations Llama would give a response in natural language only. Mistral is able to solely produce a dictionary format; however, it skips segments in some cases.

The advantage of decoder-only models over encoder-only models is immediate availability. Firstly, encoder-only models require a considerable amount of time for fine-tuning. For example, for the task of identifying requirements, BERT required about 20 min to run one epoch with a batchsize of 16 (on our dataset). Secondly, training these models requires labeled data, which are not always available.

## 5. Threats to validity

We discuss threats to validity and how we mitigated these to maximize the credibility of our study.

### 5.1. Part 1: Content analysis

We discuss the validity for RQ1 on the basis of the four categories suggested by White and Marsh [19].

*Credibility.* This is concerned with whether the collected data accurately represents the studied phenomenon.

We study projects where requirements are managed in ITSs; thus, our sampling may affect credibility. We acknowledge that a single tagger excluded irrelevant projects from the OSS datasets, and that the two taggers made the final selection for relevance based on only 12 items per project. However, the analysis of the sampled data (Table 3) indicates that the included projects are representative as they contain a significant number of requirements-related items. Moreover, per-project sampling (of 100 items per project) could affect credibility, although we mitigate this vulnerability by randomly selecting a subset of a representative size.

For some open source projects, we could not ascertain if the analyzed issues are part of a sprint or product backlog, due to the absence of such details in the datasets. We employed filtering mechanisms to only retain projects whose issue tracking systems are likely to be used to support sprint backlogs or to serve as a product backlog; however, we cannot be certain.

Furthermore, we have excluded issues labeled as "bug". Although it is possible that bug issues exist that contain requirements information, based on an initial exploration phase, we have determined that this situation is rare.

The coding scheme was created by a single tagger; then, a second tagger reviewed the data and their feedback was used to adjust the coding scheme. Finally, the feedback from a third tagger led to improving the corresponding guidelines. It is still possible that some readership would have a different understanding of the categories.

*Transferability.* This is about the extent to which the findings are transferable to other situations; in our case, to other projects where requirements are managed via ITSs.

Although we sampled fourteen diverse projects, seven of which being proprietary, the high diversity of the work practices in agile software development entails that different patterns may be found in different contexts.

For the proprietary projects, we examined projects from a single company. Selecting only projects from one specific source could lead to a number of threats. As a mitigation, we used projects from different teams and included a wide variety of OSS projects from different companies, but we reckon that future work needs to use data from multiple companies.

*Dependability.* This is concerned with the existence of an in-depth discussion of the study procedures to support replication. We try to offer a clear description of the process by describing and illustrating the research method (see Section 3.1 and Fig. 1) and by sharing the online appendix. It is possible that some details may be missing.

*Confirmability.* In quantitative content analysis, this is mostly concerned with reliability among coders. In Section 3.1.2, we report on the process we used to build the coding scheme and we present inter-rater reliability via Cohen's Kappa, which showed *almost perfect agreement*. Although this was calculated for only four projects, we decided not to use more data due to the high agreement.

### 5.2. Part 2: Experiments

To structure the threats to validity for RQ2, we follow the categories explained by Wohlin et al. [44].

*Conclusion validity.* To assess whether there is a significant difference in performance between the models in identifying and classifying requirements, we have opted for non-parametric tests: Friedman and Nemenyi's post-hoc test. Given that the sample size for these tests is equal to eight, there is a possibility that the statistical power may be affected which could introduce a threat to conclusion validity. Nevertheless, our sample size exceeds the accepted minimum of five [43].

*External validity.* In our comparison between encoder-only and decoder-only models, each model category is represented by a selection of models. Although there is a risk of sampling bias, this is mitigated by the choice of widely accepted models. In addition, for the decoder-only models, we include models from different vendors: OpenAI, Meta and Mistral AI.

*Internal validity.* We employed the GUI version of ChatGPT. Unfortunately, the GUI version does not allow setting the temperature parameter, and this may affect the model's determinism. For the other models, we mitigated the threat by setting the temperature to 0.

We did not conduct hyperparameter tuning for the encoder-only models. Consequently, it may be possible for these models to perform more optimally with other hyperparameters. To mitigate this threat, we ensured to apply widely adopted and previously tested hyperparameter values for BERT and RoBERTa [34–36]. We focused on learning rate, number of epochs, batch size, and activation function.

For decoder-only LLMs, we could not conduct fine-tuning, which would allow for a more direct comparison against encoder-only models. This task was not possible due to the excessive computational demands that go beyond our department's GPU cluster capabilities. We do, however, perform experiments with in-context learning via ChatGPT.

When performing ICL on ChatGPT, we found no significant performance improvement. Although we carefully selected appropriate demonstration examples, the lack of significant evidence may be due to the conduction of the experiment in batches of items. Performance could have been different with item-level prompting.

Furthermore, we created an additional dataset to investigate the influence of the imbalance in the dataset. To produce this *balanced dataset*, we applied random undersampling. Although we may have achieved superior performance via oversampling, the latter method can lead to overfitting [45]. Future work should, however, explore different re-balancing strategies.

Finally, the models may perform differently in dissimilar environments; to control this risk, we implement group k-fold cross-validation, where each group constitutes a project. Additionally, the 7 open-source projects served as validation sets originated from diverse providers.

*Construct validity.* The threats to construct validity would be related to the extent to which the selected metrics correctly reflect the performance of the models. To mitigate this risk, we have chosen precision, recall, and F-score, as these appear to be the most commonly reported performance metrics in software engineering research for evaluating DL and ML models [31].

## 6. Related work

We first review studies at the intersection of RE and issue tracking systems, relevant to RQ1. Next, for RQ2, we discuss the application of LLMs in RE, alongside techniques for extracting and classifying requirements.

### 6.1. Requirements in backlog items

Lüders and colleagues conducted research on the visualization and automated categorization [25] of links between issues. Our approaches are complementary. While they study the relationships between requirements and other issue types, we offer an analysis of the *contents* of the issues.

Rath et al. [46] explored the effectiveness of automated traceability by applying machine learning for recovering trace links between code commits and Jira issues. Van Oosten et al. [47] did a similar study in the context of model-driven engineering. Their focus is on repairing trace links, while we examine the requirements information in the issues.

Interview studies have been often employed to explore requirements documentation practices in agile development. For example, Behutiye et al. [48] conducted fifteen interviews with practitioners from four companies using agile software development and they studied how quality requirements are documented. They found that, in addition to using issues and epics, prototypes and face-to-face communication are essential for quality requirements. A similar analysis, Alsaqaf et al. [49] identified fifteen challenges, related to the minimal documentation

principle, in large-scale, distributed, Agile settings. Franch and colleagues [50] found that, in agile contexts, project management tools are commonly used to document requirements. Our work complements these interview studies via content analysis.

Researchers have collected and shared sets of issues extracted from publicly available repositories. The two largest and renowned datasets are the TAWOS dataset by Tawosi et al. [20] and the 'alternative' one by Montgomery and colleagues [21]. We use six projects from Montgomery's dataset and one from TAWOS, and we contrast these with seven proprietary projects.

Some studies applied content analysis to backlog items. For instance, Soliman et al. [51] investigated where architectural knowledge is located in Jira issues, Ortu and et al. [52] studied the emotions within the issues, and Montgomery et al. [21] linked issues to software lifecycle activities. We conduct a more in-depth analysis of *requirements* within backlog items.

In summary, we complement the existing studies by offering a novel, unexplored perspective: we apply (quantitative) content analysis to backlog items in ITSs, with a focus on investigating requirements' representation.

### 6.2. Extracting and classifying requirements

Researchers devised automated tools for extracting requirements, aiming to minimize manual labor. Falkner et al. [53] proposed a classifier for extracting requirements from 'Request for Proposal' documents using Naïve Bayes. Abualhaija et al. [11] used machine learning for automatically extracting (demarcating) requirements from specification document. Winkler and Vogelsang [54,55] employed neural networks to assist requirements engineers by locating requirements in specification documents, which they evaluated via a controlled experiment. These studies focus on requirements described in traditional specification documents, while we focus on requirements in agile development artifacts. Also, instead of building their models from training data, we utilize pre-trained models.

Recently, Bashir et al. [56] experimented with several models for identifying requirements within the railroad industry. They compared the performance between Machine Learning approaches, Deep Learning and pre-trained encoder-only models. Sainani et al. [57] compared a variety of models (three machine learning approaches, BiLSTM and BERT) for extracting requirements from software engineering contracts. Unlike us, these studies neither consider backlog items for extracting requirements nor include the use of decoder-only models. Wang et al. [58] conducted experiments with one decoder-only model, specifically Llama 2, to explore its use in identifying whether a sentence is a requirement or not; however, their experiments focus on traditional specification requirements.

The literature on classifying requirements is extremely rich; the mapping study by Zhao and colleagues [59] revealed over 20 tools for requirements classification that were released until April 2019, and the number only grew since then. Among the most recent approaches based on language models, Tikayati and colleagues propose a BERT-based model for classifying aerospace requirements [60], Alhoshan et al. [61] provided initial experimentation with zero-shot learning, and Hey et al. [12] apply encoder-based requirements classification for trace link recovery. We complement these endeavors by investigating the effectiveness of two families of LLMs (encoder-only and decoder-only) to the context of requirements in ITSs.

## 7. Conclusion and future work

We have addressed a knowledge problem about how requirements are formulated in ITSs (RQ1), and then explored the potential of automated techniques for identifying and classifying requirements in backlog items (RQ2). The automation aims to help practitioners locate requirements in poorly organized ITSs while minimizing intervention on team practices.

To answer RQ1 on how requirements are documented in backlog items, we first collected, tagged, and analyzed fourteen samples of open-source and proprietary projects, summing up to a total of 1,636 items. Our results show that backlog item labeling is applied inconsistently and in a misleading manner by practitioners (Finding 1.1). In addition, teams may use one backlog item to document multiple requirements, making requirements within backlogs not uniquely identifiable (Finding 1.2). Both aspects pose challenges for those who need to retrieve requirements information. Furthermore, we found that the most common categories are medium- and low-level UO requirements, which also occur together in one item and could mirror the refinement pattern of user stories into acceptance criteria (Finding 1.3). Concluding on RQ1, these findings indicate how requirements are documented in backlog items and provide evidence of the lack of an organized and systematic representation of requirements in backlog items.

To assist practitioners in locating requirements in agile settings without affecting their work methods, we examine the ability of LLMs for identifying and classifying requirements in backlog items (RQ2). For this task, we utilized the tagged dataset from our content analysis, decomposing the backlog items into segments. We compared the use of decoder-only models with encoder-only models in identifying whether a segment is a requirement or not and for classifying the requirement type. We found that encoder-only models outperform decoder-only models in identifying requirements (Findings 2.1, 2.2 and 2.3) and for classifying requirements into the UO, SO and NF categories (Findings 2.4 and 2.5). These results indicate the importance of fine-tuning a language model to the task. Although the decoder-only models are trained on a significantly larger dataset compared to BERT and RoBERTa, in our case, this did not overcome the benefits of fine-tuning on labeled data.

The results of our experiments demonstrate the complexity of identifying and classifying requirements within backlog items. Although in our experiments encoder-only models significantly outperformed decoder-only models, we encourage further research on both types of models to gain a better understanding of their performance differences. To fully learn the potential of decoder-only models, additional experimental evidence is necessary.

Future research is required to further optimize the performance on these tasks. We aim to explore additional encoder-only architectures, perform hyperparameter tuning, and experiment with different activation functions and re-sampling methods. Future work also needs to integrate an optimized model into a prototype to study its applicability and usefulness in practice.

## CRediT authorship contribution statement

**Ashley T. van Can:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Data curation, Conceptualization. **Fabiano Dalpiaz:** Writing – review & editing, Writing – original draft, Validation, Supervision, Resources, Project administration, Methodology, Investigation, Conceptualization.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgments

This research is partially funded by the Dutch Research Council (NWO) through the Open Technology Programme 2021-II TTW, project AUTOLINK (19521). We would like to express our appreciation to Mendix, and in particular to Toine Hurkmans, for providing us with the proprietary datasets used in this study. In addition, we would like to thank Marc Rademakers for helping us to improve the tagged datasets and Prof. Dr. Albert Gatt for providing valuable input regarding our experiments with in-context learning.

## Data availability

We share the link to our data/code in the paper as a reference. Regarding the data, we share data from open-source datasets (for confidentiality reasons, we exclude proprietary data)

## References

[1] D.J. Reifer, How good are agile methods? IEEE Softw. 19 (4) (2002) 16–18.
[2] K. Beck, M. Beedle, A. van Bennekum, A. Cockburn, W. Cunningham, M. Fowler, J. Grenning, J. Highsmith, A. Hunt, R. Jeffries, J. Kern, B. Marick, R.C. Martin, S. Mellor, K. Schwaber, J. Sutherland, D. Thomas, Manifesto for agile software development, 2001, Web page, https://agilemanifesto.org/ (Accessed 1 July 2024).
[3] F. Dalpiaz, J.-P. Steghöfer, Where requirements and agility meet: No-man's-land or a land of opportunity? IEEE Softw. 41 (3) (2024) 7–10.
[4] S. Bühne, A. Herrmann, Advanced level requirements management - handbook, in: Certified Professional for Requirements Engineering, CPRE, 2024, Version 2.1.0.
[5] I. Inayat, S.S. Salim, S. Marczak, M. Daneva, S. Shamshirband, A systematic literature review on agile requirements engineering practices and challenges, Comput. Hum. Behav. 51 (2015) 915–929.
[6] T. Sedano, P. Ralph, C. Péraire, The product backlog, in: Proc. of ICSE, IEEE, 2019, pp. 200–211.
[7] A.T. van Can, F. Dalpiaz, Requirements information in backlog items: Content analysis, in: Proc. of REFSQ, in: LNCS, vol. 14588, Springer, 2024, pp. 305–321.
[8] M. Glinz, A glossary of requirements engineering terminology, in: Standard Glossary of the Certified Professional for Requirements Engineering (CPRE) Studies and Exam, Version 2.0.1, 2022.
[9] G. Lucassen, F. Dalpiaz, J.M.E. van der Werf, S. Brinkkemper, Improving agile requirements: the quality user story framework and tool, Requir. Eng. 21 (3) (2016) 383–403.
[10] G. Lucassen, F. Dalpiaz, J.M. van der Werf, S. Brinkkemper, Improving user story practice with the Grimm method: A multiple case study in the software industry, in: Proceedings of the 23rd International Working Conference on Requirements Engineering: Foundation for Software Quality, REFSQ'17, in: LNCS, vol. 10153, 2017, pp. 235–252.
[11] S. Abualhaija, C. Arora, M. Sabetzadeh, L.C. Briand, M. Traynor, Automated demarcation of requirements in textual specifications: a machine learning-based approach, Empir. Softw. Eng. 25 (2020) 5454–5497.
[12] T. Hey, J. Keim, S. Corallo, Requirements classification for traceability link recovery, in: Proc. of RE, IEEE, 2024.
[13] IEEE, Systems and software engineering – life cycle processes –requirements engineering, in: ISO/IEC/IEEE 29148:2018(E), 2018.
[14] E. Yu, J. Mylopoulos, Understanding "why" in software process modelling, analysis, and design, in: Proc. of ICSE, IEEE, 1994, pp. 159–168.
[15] W.X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, et al., A survey of large language models, 2023, arXiv preprint arXiv:2303.18223.
[16] J. Yang, H. Jin, R. Tang, X. Han, Q. Feng, H. Jiang, S. Zhong, B. Yin, X. Hu, Harnessing the power of LLMs in practice: A survey on ChatGPT and beyond, ACM Trans. Knowl. Discov. Data (160) (2023).
[17] A. Vogelsang, J. Fischbach, Using large language models for natural language processing tasks in requirements engineering: A systematic guideline, 2024, arXiv preprint arXiv:2402.13823.
[18] K. Krippendorff, Content Analysis: An Introduction to Its Methodology, Sage publications, 2018.
[19] M.D. White, E.E. Marsh, Content analysis: A flexible methodology, Libr. Trends 55 (1) (2006) 22–45.
[20] V. Tawosi, A. Al-Subaihin, R. Moussa, F. Sarro, A versatile dataset of agile open source software projects, in: Proc. of MSR, 2022, pp. 707–711.
[21] L. Montgomery, C. Lüders, W. Maalej, An alternative issue tracking dataset of public Jira repositories, in: Proc. of MSR, 2022, pp. 73–77.
[22] M. Cohn, User Stories Applied: for Agile Software Development, Addison-Wesley Professional, 2004.
[23] M. Kassab, The changing landscape of requirements engineering practices over the past decade, in: Proc. of EmpiRE, IEEE, 2015, pp. 1–8.
[24] G. Lucassen, F. Dalpiaz, J.M. van der Werf, S. Brinkkemper, The use and effectiveness of user stories in practice, in: Proc. of REFSQ, in: LNCS, vol. 9619, 2016, pp. 205–222.
[25] C.M. Lüders, T. Pietz, W. Maalej, On understanding and predicting issue links, Requir. Eng. (2023) 1–25.
[26] Ashley T. van Can and Fabiano Dalpiaz, Online Appendix of the Paper: 'Locating Requirements in Backlog Items: Content Analysis and Experiments with Large Language Models'", 2024, https://doi.org/10.5281/zenodo.14445642.
[27] J. Cohen, A coefficient of agreement for nominal scales, Educ. Psychol. Meas. 20 (1) (1960) 37–46.
[28] L.M. Cysneiros, J.C.S. do Prado Leite, J. de Melo Sabat Neto, A framework for integrating non-functional requirements into conceptual models, Requir. Eng. 6 (2001) 97–115.
[29] D. Gross, E. Yu, From non-functional requirements to design through patterns, Requir. Eng. 6 (2001) 18–36.
[30] R.E. Jeffries, A. Anderson, C. Hendrickson, Extreme Programming Installed, Addison-Wesley Longman Publishing Co. Inc., Boston, MA, USA, 2000.
[31] D. Dell'Anna, F.B. Aydemir, F. Dalpiaz, Evaluating classifiers in SE research: The ECSER pipeline and two replication studies, Empir. Softw. Eng. 28 (3) (2023).
[32] J.D. Rodriguez, A. Perez, J.A. Lozano, Sensitivity analysis of k-fold cross validation in prediction error estimation, IEEE Trans. Pattern Anal. Mach. Intell. 32 (3) (2009) 569–575.
[33] A. Fernández, V. López, M. Galar, M.J. del Jesus, F. Herrera, Analysing the classification of imbalanced data-sets with multiple classes: Binarization techniques and ad-hoc approaches, Knowl.-Based Syst. 42 (2013) 97–110.
[34] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, BERT: Pre-training of deep bidirectional transformers for language understanding, 2018, arXiv preprint arXiv:1810.04805.
[35] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, V. Stoyanov, RoBERTa: A robustly optimized bert pretraining approach, 2019, arXiv preprint arXiv:1907.11692.
[36] H. Sobhanam, J. Prakash, Analysis of fine tuning the hyper parameters in RoBERTa model using genetic algorithm for text classification, Int. J. Inf. Technol. 15 (7) (2023) 3669–3677.
[37] A. Krizhevsky, I. Sutskever, G.E. Hinton, Imagenet classification with deep convolutional neural networks, Adv. Neural Inf. Process. Syst. 25 (2012).
[38] S.R. Dubey, S.K. Singh, B.B. Chaudhuri, Activation functions in deep learning: A comprehensive survey and benchmark, Neurocomputing 503 (2022) 92–108.
[39] Q. Dong, L. Li, D. Dai, C. Zheng, Z. Wu, B. Chang, X. Sun, J. Xu, Z. Sui, A survey on in-context learning, 2022, arXiv preprint arXiv:2301.00234.
[40] J. Liu, D. Shen, Y. Zhang, B. Dolan, L. Carin, W. Chen, What makes good in-context examples for GPT-3 ? 2021, arXiv preprint arXiv:2101.06804.
[41] J. White, Q. Fu, S. Hays, M. Sandborn, C. Olea, H. Gilbert, A. Elnashar, J. Spencer-Smith, D.C. Schmidt, A prompt pattern catalog to enhance prompt engineering with ChatGPT, 2023, arXiv preprint arXiv:2302.11382.
[42] OpenAI, Prompt engineering, 2023, https://platform.openai.com/docs/guides/prompt-engineering?ref=blef.fr (Accessed 20 December 2023).
[43] J. Demšar, Statistical comparisons of classifiers over multiple data sets, J. Mach. Learn. Res. 7 (2006) 1–30.
[44] C. Wohlin, P. Runeson, M. Höst, M.C. Ohlsson, B. Regnell, A. Wesslén, et al., Experimentation in Software Engineering, vol. 236, Springer, 2012.
[45] H. He, E.A. Garcia, Learning from imbalanced data, IEEE Trans. Knowl. Data Eng. 21 (9) (2009) 1263–1284.
[46] M. Rath, J. Rendall, J.L. Guo, J. Cleland-Huang, P. Mäder, Traceability in the wild: Automatically augmenting incomplete trace links, in: Proc. of ICSE, 2018, pp. 834–845.
[47] W. van Oosten, R. Rasiman, F. Dalpiaz, T. Hurkmans, On the effectiveness of automated tracing from model changes to project issues, Inf. Softw. Technol. 160 (2023) 107226.
[48] W. Behutiye, P. Seppänen, P. Rodríguez, M. Oivo, Documentation of quality requirements in agile software development, in: Proc. of EASE, 2020, pp. 250–259.
[49] W. Alsaqaf, M. Daneva, R. Wieringa, Quality requirements challenges in the context of large-scale distributed agile: An empirical study, Inf. Softw. Technol. 110 (2019) 39–55.
[50] X. Franch, C. Palomares, C. Quer, P. Chatzipetrou, T. Gorschek, The state-of-practice in requirements specification: An extended interview study at 12 companies, Requir. Eng. (2023) 1–33.
[51] M. Soliman, M. Galster, P. Avgeriou, An exploratory study on architectural knowledge in issue tracking systems, in: Proc. of ECSA, Springer, 2021, pp. 117–133.
[52] M. Ortu, A. Murgia, G. Destefanis, P. Tourani, R. Tonelli, M. Marchesi, B. Adams, The emotional side of software developers in JIRA, in: Proc. of MSR, 2016, pp. 480–483.
[53] A. Falkner, C. Palomares, X. Franch, G. Schenner, P. Aznar, A. Schoerghuber, Identifying requirements in requests for proposal: A research preview, in: Proc. of REFSQ, in: LNCS, vol. 11412, Springer, 2019, pp. 176–182.

[54] J. Winkler, A. Vogelsang, Automatic classification of requirements based on Convolutional Neural Networks, in: Workshop Proceedings of RE, IEEE, 2016, pp. 39–45.
[55] J.P. Winkler, A. Vogelsang, Using tools to assist identification of non-requirements in requirements specifications–A controlled experiment, in: Proc. of REFSQ, in: LNCS, vol. 10753, Springer, 2018, pp. 57–71.
[56] S. Bashir, M. Abbas, M. Saadatmand, E.P. Enoiu, M. Bohlin, P. Lindberg, Requirement or not, that is the question: A case from the railway industry, in: Proc. of REFSQ, in: LNCS, vol. 13975, Springer, 2023, pp. 105–121.
[57] A. Sainani, P.R. Anish, V. Joshi, S. Ghaisas, Extracting and classifying requirements from software engineering contracts, in: Proc. of RE, IEEE, 2020, pp. 147–157.
[58] K. Wang, F. Zhang, M. Sabetzadeh, Automated requirements demarcation using large language models: An empirical study, in: Workshop Proceedings of REFSQ, 2024.
[59] L. Zhao, W. Alhoshan, A. Ferrari, K.J. Letsholo, M.A. Ajagbe, E.-V. Chioasca, R.T. Batista-Navarro, Natural language processing for requirements engineering: A systematic mapping study, ACM Comput. Surv. 54 (3) (2021) 1–41.
[60] A. Tikayat Ray, B.F. Cole, O.J. Pinon Fischer, R.T. White, D.N. Mavris, AeroBERT-classifier: Classification of aerospace requirements using bert, Aerosp. 10 (3) (2023) 279.
[61] W. Alhoshan, A. Ferrari, L. Zhao, Zero-shot learning for requirements classification: An exploratory study, Inf. Softw. Technol. 159 (2023) 107202.